

HackMate — Project Defense & Viva Handbook

In-Depth Technical Mastery Guide & Counter-Question Survival Handbook

Target Audience: Specially designed for team members from non-technical backgrounds to speak with confidence, explain their assigned module in plain English, and answer tough professor counter-questions accurately.

■ The 3 Golden Rules for Non-Tech Presenters:

- 1. Never Panic:** You do not need to write raw code on the board. You only need to explain *what it does, why we chose it, and how it helps the student*.
- 2. Use Real Examples:** When asked about database or frontend, give the SIH / Hackathon example (e.g., 'A team needs 1 React Dev and 1 ML Specialist').
- 3. The Smooth Handover Rule:** If a professor asks a super-deep backend architecture or low-level memory question, say: *'I handled the module integration and security flow, while Member 1 (Lead) optimized the low-level concurrency engine.'*

PART 1: MEMBER 2 GUIDE — Frontend UI/UX & Animations

Assigned Role:	Frontend & UI/UX Developer (React 18, Tailwind CSS, GSAP)
Simple Analogy:	If HackMate is a car, the Frontend is the dashboard, steering wheel, comfortable seats, and smooth paint job that the driver actually sees and touches.
Your Deliverables:	React Component Islands (`TeamCard`, `FilterBar`, `Navbar`, `Modals`), Tailwind responsive styling, GSAP entrance and modal animations.

Your 45-Second Presentation Script:

"I developed the **Frontend User Interface and interactive animations** for HackMate.

- I designed modular **React 18 components** such as the TeamCard (which displays live roster progress bars and skill tags), the FilterBar for instant keyword filtering, and popup modals for creating squads and submitting pitches.
- I styled the whole interface using **Tailwind CSS**, using a clean, modern color scheme with high-contrast badge tags so students can scan needed roles in seconds.
- I integrated **GSAP (GreenSock)** to create smooth entrance timelines, spring tab switching on the login screen, and interactive modal transitions that feel snappy and professional."

Professor Counter-Questions & Perfect Answers for Member 2:

Q1: What is a React Component, and why did you use React instead of plain HTML?

Answer: A React component is like a reusable Lego block. Instead of copying and pasting the HTML code for a team card 50 times, we created one TeamCard.jsx component. Whenever the backend gives us a list of 10 squads, React automatically renders 10 cards with their respective data. If we want to change the button color or card border, we edit it once in TeamCard.jsx and all 10 cards update instantly.

Q2: Why did you use Tailwind CSS instead of writing a standard CSS file?

Answer: Regular CSS files become messy, bloated, and hard to maintain across large projects. Tailwind CSS uses utility classes directly inside our components (like flex, rounded-2xl, bg-terracotta). This ensures 100% consistent spacing, typography, and responsive layouts for mobile and desktop without writing thousands of lines of custom CSS.

Q3: What does GSAP do on your website?

Answer: GSAP (GreenSock Animation Platform) handles smooth micro-animations. Specifically, it powers the entrance animations when the page loads, the floating status badges on the hero image, and the spring physics when switching between Sign In and Registration tabs so the user experience feels modern and fluid.

Q4: How does the frontend send data to the backend?

Answer: We built a centralized helper file called `api.js` that uses the standard JavaScript `fetch()` API. When a student clicks 'Apply to Squad', `api.js` converts their pitch message into a JSON object, attaches their security JWT token to the request header, and sends an HTTP POST request to our Java REST backend at `/api/requests`.

Q5: What happens if a user submits a form with missing information?

Answer: The frontend performs instant client-side validation. Required inputs have HTML5 required tags, and our React state checks for empty fields before calling the API, displaying a clean alert banner immediately so the server is not flooded with bad requests.

PART 2: MEMBER 3 GUIDE — Database Architecture & MongoDB

Assigned Role:	Database Engineer & Data Modeler (MongoDB 7.0, Java Sync Driver)
Simple Analogy:	If HackMate is a university office, MongoDB is a flexible digital filing cabinet. Instead of rigid excel sheets where columns can't change, each student and team has a digital folder containing all their details and dynamic skills together.
Your Deliverables:	MongoDB collection schemas (`users`, `teams`, `join_requests`), polymorphic skill array modeling, atomic `\$push` roster updates, and zero-config demo seed data.

Your 45-Second Presentation Script:

"I designed the **Database Architecture and Data Modeling using MongoDB**.

1. I structured our persistence layer into three primary collections: *users* for verified student profiles, *teams* for hackathon squad postings, and *join_requests* for managing collaboration pitches.
2. We specifically selected **MongoDB (NoSQL)** because hackathon teams require flexible arrays of skills—like Python, React, PyTorch, or Solidity—and embedded lists of team members. MongoDB allows us to store and query these arrays natively and perform atomic roster updates using *\$push* without expensive multi-table SQL joins.
3. I also configured our database seed loader so that when the server boots up, pre-configured student accounts and sample hackathon squads are populated for instant testing."

Professor Counter-Questions & Perfect Answers for Member 3:

Q1: (MOST IMPORTANT) Why did you choose MongoDB over MySQL or a Relational Database?

Answer: Hackathon team discovery data is naturally document-oriented and polymorphic. Each team post requires an arbitrary list of skills (e.g. ['Python', 'FastAPI', 'PyTorch']) and an embedded array of accepted members with specific roles. In MongoDB, an entire squad with its members and skills is stored in a single JSON-like document. In MySQL, storing dynamic skills and rosters would require 4 separate normalized tables (*teams*, *skills*, *team_skills*, *team_members*) joined together on every query, which creates unnecessary complexity and relational join overhead for a web-scale listing platform.

Q2: What is the primary key in MongoDB collections?

Answer: In MongoDB, the primary key is the `_id` field, which is an automatically generated 12-byte ObjectId. It guarantees global uniqueness across distributed systems and contains a timestamp of when the document was created.

Q3: How does the database handle adding a new member to an existing team?

Answer: When a squad leader accepts an applicant, we execute MongoDB's native *\$push* atomic operator to append the new member object { `userId`, `name`, `role` } directly into the team's `currentMembers` array. Because it is atomic, it prevents duplicate member additions or race conditions.

Q4: How do you filter teams by a specific skill like 'React' or 'Python'?

Answer: In MongoDB, since `skillsNeeded` is an array of strings, we query it using case-insensitive regex or exact string matching using the MongoDB Java Driver's `Filters.regex('skillsNeeded', ...)`. MongoDB searches inside the array elements automatically without needing a join table.

Q5: What happens if MongoDB is not installed on the professor's computer when evaluating?

Answer: We integrated an embedded in-memory wire-protocol server called `mongo-java-server` in our Maven configuration. If the backend detects that no local MongoDB daemon is running on port 27017, it automatically starts the in-memory fallback server seamlessly with zero manual configuration needed.

PART 3: MEMBER 4 GUIDE — Authentication & Security Pipeline

Assigned Role:	Security & Auth Engineer (BCrypt Hashing, JWT HMAC256, Javalin AuthFilter)
Simple Analogy:	If HackMate is an exclusive campus hackathon event, BCrypt is a secure vault that scrambles your locker key, and JWT is a digital, tamper-proof VIP wristband that lets you enter team rooms without having to show your ID card at every single door.
Your Deliverables:	BCrypt password salting and hashing, JWT HMAC256 token generation and verification, Javalin `AuthFilter` route guards, and secure client session handling.

Your 45-Second Presentation Script:

"I developed the **Authentication and Security Pipeline** for HackMate.

1. When students register, we never store plain-text passwords. Instead, I used **BCrypt with salt factor 10** to generate a one-way cryptographic hash before saving to MongoDB.
2. For user sessions, we implemented **JWT (JSON Web Tokens)** signed using the **HMAC256 algorithm**. When a student logs in, the server generates a token containing their user ID and email with a secure 24-hour expiration window.
3. On the backend, I implemented a global **AuthFilter middleware** in Javalin. It intercepts all incoming requests to protected routes like `/api/teams` and `/api/requests`, verifies the Bearer token signature, and extracts the authenticated student context automatically."

Professor Counter-Questions & Perfect Answers for Member 4:

Q1: Why did you use JWT instead of traditional server-side Sessions?

Answer: Traditional session-based authentication stores session IDs in server memory. If thousands of students use the platform or if we scale across multiple servers, the server runs out of memory or requires complex session sharing. JWT is completely **stateless**—all identity data is cryptographically signed inside the token itself, so the server only needs to verify the HMAC256 signature without querying session memory.

Q2: Can a malicious user tamper with a JWT token to impersonate someone else?

Answer: No. A JWT token consists of three parts: Header, Payload, and Signature. The Signature is generated using our secret key on the server via HMAC256. If a hacker alters even a single character in the payload (like changing their user ID), the signature verification mathematically fails, and our AuthFilter immediately rejects the request with HTTP 401 Unauthorized.

Q3: Why can't we decrypt a BCrypt password to see what it was?

Answer: BCrypt is a **one-way mathematical hash function**, not encryption. You cannot reverse a hash back to plaintext. When the user logs in again, BCrypt hashes their entered password with the stored salt and compares the two hashes using `BCrypt.checkpw()`. Even if someone accesses the database, the original passwords cannot be recovered.

Q4: How does the Javalin backend know which user created a squad?

Answer: When a request reaches Javalin, our AuthFilter extracts the Bearer token from the HTTP Authorization header, verifies it with `JwtUtil`, and attaches the student's `userId` to Javalin's `ctx.attribute('userId')`. The `TeamController` then reads this attribute directly, guaranteeing that the squad's `createdBy` field is authentic and cannot be spoofed.

Q5: What happens when the JWT token expires after 24 hours?

Answer: The JWT library automatically checks the token's expiration claim (`exp`). If the token is older than 24 hours, verification fails, Javalin returns HTTP 401 Unauthorized, and our frontend `api.js` catches this, clears the `localStorage`, and redirects the user to `login.html`.

PART 4: Master Project Summary & Universal Cheat Sheet

The 30-Second Elevator Pitch (Anyone can say this!):

"HackMate is a specialized campus hackathon squad formation hub. Instead of chaotic, noisy WhatsApp groups with unread message floods and ghosting teammates, HackMate allows collegiate developers to filter open teams by exact tech stack (React, Python, Solidity), view real-time roster progress bars, and send 1-click structured application pitches with verified GitHub portfolios."

End-to-End Data Flow (How all 4 parts connect together):

Step	Component / Layer	Exact Action Occurring
1	Frontend (React/Vite)	Student selects skill filter or fills 'Apply to Squad' modal with their pitch.
2	Auth Client (api.js)	Injects `Authorization: Bearer ` header into the HTTP POST request.
3	Backend Filter (Javalin)	`AuthFilter` validates HMAC256 signature and binds `userId` to the request context.
4	Controller & Service	`TeamController` verifies squad capacity is not full and formats document.
5	Database (MongoDB)	Executes atomic `\$push` operator to save request in `join_requests` collection.
6	UI Update (GSAP)	Returns JSON response; React updates roster meter and GSAP animates status badge.

Emergency Panic Protocol (If you get stuck on a question):

- **If asked about code you don't know:** Say: *'In our architecture, I focused on the functional design and API integration for this module, while Member 1 structured the underlying controller logic.'*
- **If asked 'Why not use SQL / Relational?':** Say: *'Because hackathon teams have dynamic lists of skills and rosters that vary per post. Storing them as JSON documents in MongoDB avoids 4-table SQL joins on every query.'*
- **If asked 'Is it secure?':** Say: *'Yes, passwords use BCrypt hashing with salt rounds, and all protected endpoints require HMAC256-signed JWT tokens.'*